

Project Documentation

RISC-V Canny Edge Detection Pipeline - Version 2.0

1. Project Overview

This project implements a Canny Edge Detection pipeline in C++, specifically optimized for and cross-compiled to the **RISC-V (RV64GCV)** architecture. The project follows a strict Test-Driven Development (TDD) methodology, ensuring mathematical correctness before deploying to the target hardware via QEMU emulation. The ultimate goal is to benchmark and optimize the image processing algorithms using hardware-specific features, such as the RISC-V Vector Extension (**V**).

2. Architecture & Pipeline Components

The software has been expanded from a filtering baseline to a complete computer vision pipeline. All stages are implemented in scalar C++:

- **Gaussian Blur** (`gaussian.h`): Applies a 5x5 convolution kernel to smooth the image [cite: gaussian.h].
- **Sobel Filter** (`sobel.h`): Computes the spatial gradient of the image using 3x3 kernels.
- **Gradient Magnitude** (`magnitude.h`): Combines gradients using L1/L2 norms and normalizes output to [0, 255] [cite: magnitude.h].
- **Gradient Direction** (`direction.h`): Quantizes edges into four primary bins (0°, 45°, 90°, 135°).
- **Non-Max Suppression** (`nms.h`): Thins edges by comparing magnitudes along the gradient direction [cite: nms.h].
- **Double Thresholding** (`threshold.h`): Classifies pixels as STRONG, WEAK, or NON-EDGE; includes `auto_threshold` logic [cite: threshold.h].
- **Hysteresis** (`hysteresis.h`): Uses a stack-based flood fill to finalize edges [cite: hysteresis.h].

3. Development Phases & Milestones

Phase 1 - 3: Foundation and Verification

Environment setup and scalar implementations are complete. All modules, including the new NMS and Hysteresis stages, have accompanying unit tests in `test_pipeline.cpp` to ensure algorithmic integrity across all pipeline additions.

Phase 4: Compiler Optimization Sweep

Updated Status: This phase is currently being updated to include the full 7-stage pipeline.

The introduction of `nms.h`, `threshold.h`, and `hysteresis.h` adds branch-heavy logic and standard library dependencies (`std::stack`) that require a fresh benchmark across all optimization levels (`-O0`, `-O2`, `-O3`, `-Os`, `-Ofast`).

Phase 5: Profiling & Bottleneck Analysis

Updated Status: Phase 5 profiling is being re-run.

Initial profiling verified the first four stages. The timing harness in `main.cpp` is being expanded to measure the millisecond execution time of the **entire** pipeline, specifically identifying how much cycle time is consumed by the new non-convolutional stages [cite: main.cpp].

4. Build System & Usage

The project utilizes a `Makefile` to manage compilation for both the native host and the RISC-V target.

Host Machine Testing

Validates mathematical logic using Google Test:

```
make clean
make test
```

RISC-V Emulation

Cross-compiles and runs on the QEMU emulator:

```
make clean
make run
```

Optimization Sweeps

Benchmarks full pipeline performance:

```
make run OPT=-O3
```

5. Current Status & Next Steps

Status: The full scalar pipeline is mathematically verified. Phase 4 and 5 are being updated to provide a complete performance baseline for all 7 modules.

Upcoming Milestone (Phase 6 - Vectorization):

Once the updated profiling identifies the highest-latency modules, the project will move to writing explicit SIMD vector intrinsic functions (`<riscv_vector.h>`) to accelerate the bottlenecked components.